

ECOM EXPERTS

Ecom NoCookies

Manual de usuario e instalación

ecom_nocookies 1.2.1 · 04/09/2026

Contenido

1. Qué hace y para quién

2. Lo que tienes que saber antes de usarla (importante)

3. Compatibilidad

4. Instalación

Qué permisos pide, y por qué

5. Cómo se usa, día a día

La ventana emergente

Los dos botones

Los muros «consentir o pagar»

El contador

6. Preguntas frecuentes y problemas

7. Seguridad: qué se ha revisado y qué se ha corregido

Corregido

Revisado y correcto de origen

Pendiente, y por qué no se ha tocado

8. Ficha técnica

Ficheros

Permisos declarados

Cómo encaja todo

Configuración guardada (chrome.storage.local)

Rechazo por API, por CMP

Añadir un banner que no se rechaza

Sin registros

Prueba automática

Extensión de navegador que rechaza automáticamente los avisos de cookies y te dice, sitio a sitio, en qué estado ha quedado tu consentimiento.

1. Qué hace y para quién

Casi todas las webs abren con un cartel de cookies. Aceptarlo es un clic; rechazarlo suele costar dos o tres, y a menudo el botón de rechazar está escondido detrás de «Configurar preferencias». La consecuencia práctica es que la mayoría de la gente acaba aceptando por cansancio. **Ecom NoCookies pulsa por ti el botón de rechazar**, en cuanto el aviso aparece, sin que tengas que verlo.

Va más allá de esconder el cartel. Ocultar el aviso con un bloqueador de anuncios deja la web esperando una respuesta que nunca llega, y muchos sitios se quedan con el valor que traían por defecto. Esta extensión **interactúa de verdad** con el gestor de consentimiento (el CMP): llama a `Cookiebot.decline()`, a `OneTrust.RejectAll()`, a `Didomi.setUserDisagreeToAll()` y equivalentes, o pulsa el botón real del banner. La web se entera de tu negativa y la registra.

La segunda mitad del trabajo es **informarte**. La ventana emergente te dice qué gestor de consentimiento usa la página, si tiene Google Consent Mode activo y, categoría por categoría (`ad_storage`, `analytics_storage`, `ad_user_data`, `ad_personalization` ...), si el permiso está concedido o denegado. Eso convierte la extensión en una herramienta de auditoría: sirve para comprobar si la tienda de un cliente respeta de verdad el rechazo del visitante, o si sigue cargando analítica y publicidad pese a haber dicho que no.

Está pensada para dos perfiles: quien simplemente quiere navegar sin dar datos y sin pelearse con carteles, y quien trabaja con tiendas y necesita verificar que el aviso de cookies de un sitio hace lo que dice que hace.

2. Lo que tienes que saber antes de usarla (importante)

Esto tiene consecuencias legales y conviene leerlo entero.

La extensión toma una decisión en tu nombre. No se limita a tapar el cartel: **expresa un rechazo**, y el gestor de consentimiento lo guarda como una elección tuya, con su fecha y su hora, exactamente igual que si hubieras pulsado el botón con el ratón. Ante la web, y ante un eventual registro de consentimiento, ese rechazo eres tú.

Nunca acepta nada. La extensión solo pulsa botones de rechazo. No hay ninguna ruta del código que pulse «Aceptar», «Aceptar todo» ni «Permitir». Se ha comprobado en la prueba automática: con el botón «Aceptar todo» delante, no lo toca. Si algún día no encuentra un botón de rechazo, no hace nada; jamás acepta «para quitarlo de en medio».

Rechazar es lo prudente, pero no es neutro. Rechazar es la opción que menos datos cede, y bajo el RGPD equivale a la ausencia de consentimiento. Aun así, en algunos sitios rechazar tiene efectos: se pierden preferencias que se guardaban en cookies (el idioma, la moneda, el carrito en tiendas mal montadas) o se recorta alguna función. Si un sitio concreto te interesa que funcione tal cual, ponlo en la lista blanca.

Muros «consentir o pagar». Cada vez más medios ofrecen «acepta la publicidad o paga una suscripción». Ahí rechazar **te manda a la pasarela de pago o te deja fuera del contenido**. Por eso la extensión los detecta y, por defecto, **no rechaza** en ellos: te avisa en la ventana emergente y te deja decidir. Solo si tú activas «Rechazar aquí de todas formas» actuará en ese sitio.

Cuando solo puede ocultar. Si no encuentra el botón de rechazo pero sí el cartel, puede acabar ocultándolo. En ese caso **no se ha registrado ninguna decisión**: para el sitio sigues sin haber contestado. Legalmente eso es «sin consentimiento», que es el lado seguro, pero puede que el aviso te reaparezca en la siguiente visita.

No sustituye a un asesor. Es una herramienta de navegación y de diagnóstico. Que la ventana emergente diga «Cookies rechazadas» no certifica que el sitio cumpla la normativa, solo que se ha pulsado el botón de rechazar y qué ha declarado la web después.

3. Compatibilidad

Navegadores	Google Chrome, Microsoft Edge, Brave, Opera y cualquier otro basado en Chromium
Versión mínima	Chrome/Edge 111 , declarada en el manifiesto
Manifiesto	Manifest V3
Firefox / Safari	No compatibles (no soportan <code>world: "MAIN"</code> en los content scripts igual)
Idioma de la interfaz	Castellano
Sistema operativo	Cualquiera con un navegador Chromium (Windows, macOS, Linux)

El requisito de la versión 111 no es opcional: la extensión necesita ejecutar un script en el «mundo MAIN» de la página, que es la única forma de leer `window.dataLayer` y los objetos del gestor de consentimiento. Esa capacidad se añadió en Chrome 111.

Gestores de consentimiento con soporte por API (rechazo limpio, mediante su propia función): Cookiebot, OneTrust, Didomi, Usercentrics, Osano, Complianz y CookiesPlus.

Detectados e informados (rechazo por botón, no por API): Quantcast, IAB TCF, CookieYes, Consentiam, Cookie Notice y Borlabs.

Cualquier otro banner se intenta por selectores conocidos y por el texto del botón.

4. Instalación

La extensión se entrega como carpeta, sin publicar en la Chrome Web Store, así que se instala en modo desarrollador:

1. Abre `chrome://extensions` (en Edge, `edge://extensions`).
2. Activa el interruptor **Modo desarrollador**, arriba a la derecha.
3. Pulsa **Cargar descomprimida**.
4. Selecciona la carpeta `ecom_nocookies` completa (la que contiene `manifest.json`).
5. Aparecerá «Ecom NoCookies 1.2.1». Ancla su icono a la barra con el botón del puzle.

No crea ningún fichero fuera del navegador, no instala servicios ni tareas programadas y no requiere permisos de administrador. Para desinstalarla, **Quitar** en esa misma pantalla: se borra también su configuración (interruptor, lista blanca y contador).

Qué permisos pide, y por qué

Permiso	Para qué
<code>storage</code>	Guardar tu configuración: interruptor general, lista blanca y contador. Solo en tu navegador.
<code>activeTab</code>	Saber en qué dominio estás cuando abres la ventana emergente.
Acceso a todos los sitios	Los avisos de cookies aparecen en cualquier web, así que sus scripts tienen que poder ejecutarse en todas.

No pide `cookies`, ni `tabs`, ni `scripting`, ni `webRequest`, ni `declarativeNetRequest`. No lee ni escribe tus cookies directamente: solo pulsa botones en la propia página, igual que harías tú.

No envía nada a ningún servidor. Se ha revisado el código entero: no hay una sola llamada de red (`fetch`, `XMLHttpRequest`, ni ninguna otra). El contador y la lista blanca viven en `chrome.storage.local`, dentro de tu navegador. No hay analítica, ni telemetría, ni cuentas de usuario.

5. Cómo se usa, día a día

El uso normal es **no usarla**: se instala y trabaja sola. Cuando entras en una web con aviso de cookies, lo rechaza en el primer segundo y el icono de la extensión se marca con un **✓ verde**. Ese visto es la señal de que ha actuado en esa pestaña.

Aquí, la extensión ha rechazado el aviso de una página de prueba: el cartel y su fondo oscuro han desaparecido, y los botones propios del sitio siguen intactos.

Tienda de prueba

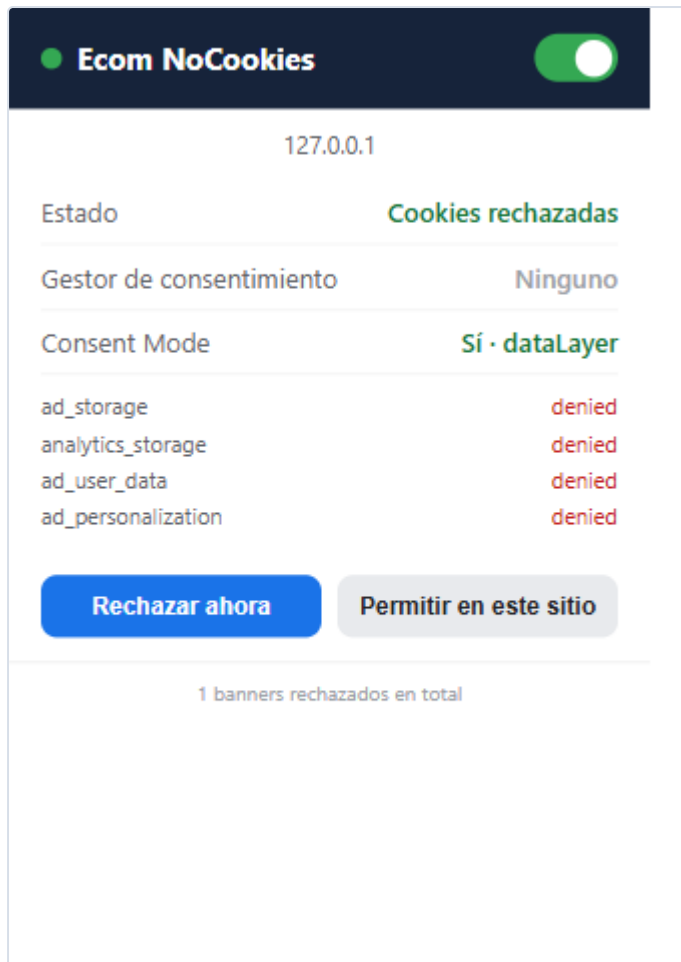
Página de ejemplo con un aviso de cookies casero y Google Consent Mode simulado, para comprobar el comportamiento de la extensión.

Zona del sitio (nada que ver con cookies). Estos botones NO se deben tocar:

```
{"rechazado":true,"aceptado":false,"sitioTocado":[]}
```

La ventana emergente

Al pulsar el icono se abre el panel de estado del sitio en el que estés:



De arriba abajo:

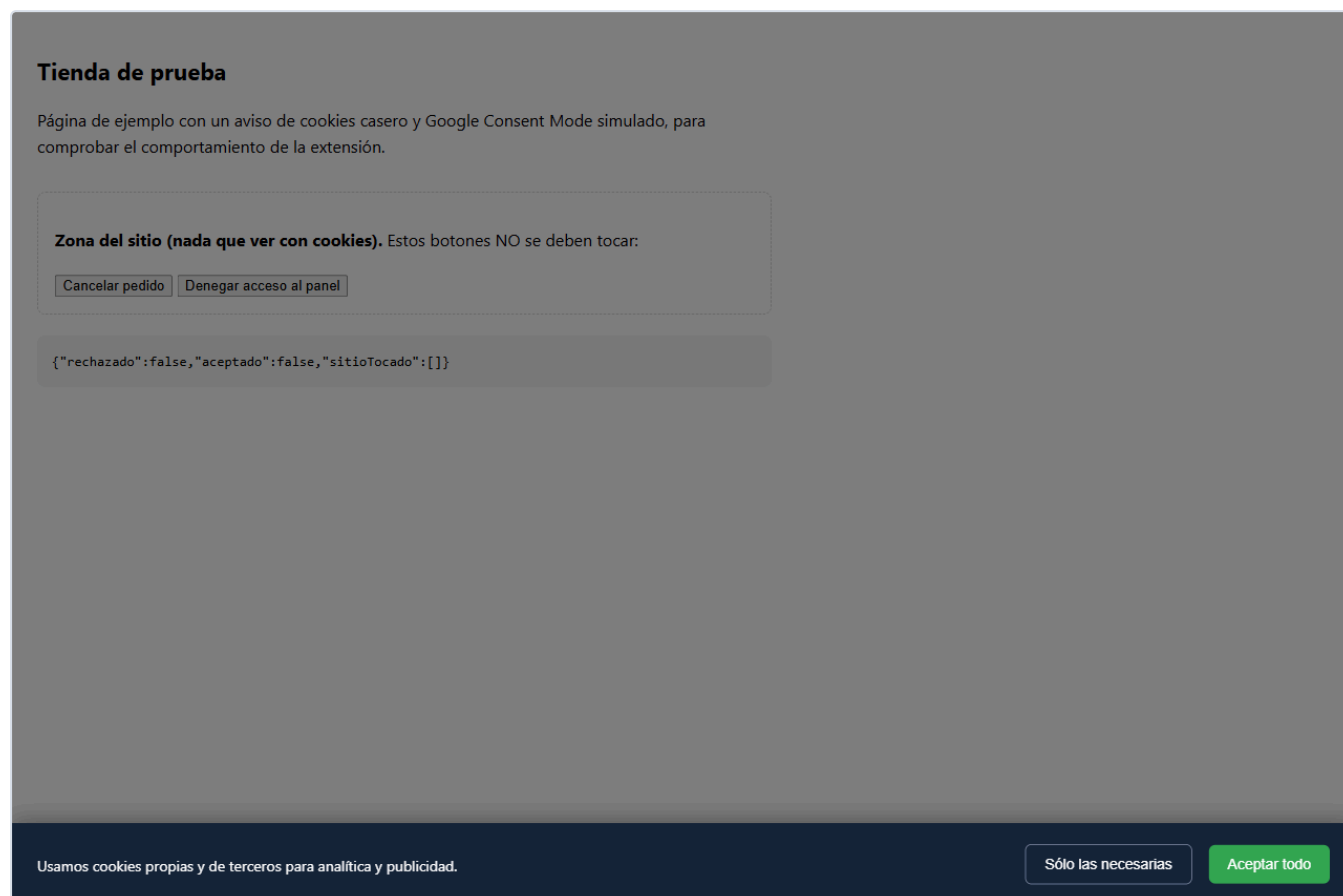
- **Interruptor verde (arriba a la derecha).** Apaga o enciende la extensión **en todas las webs**. Apagada, no toca ningún banner. El cambio se aplica en la siguiente carga de página.
- **El dominio**, centrado bajo la cabecera. Es el sitio al que se refiere todo lo demás.
- **Estado.** El resultado en esta página:
- *Cookies rechazadas* — ha pulsado el rechazo. Es el caso normal.
- *Banner detectado (sin rechazar)* — ha encontrado un gestor de consentimiento pero todavía no ha podido pulsar nada. Suele resolverse solo en un instante; si se queda así, ese banner necesita que se le añada su selector.
- *Muro consentir-o-pagar* — ha reconocido un muro de pago y **se ha detenido a propósito**. Ver más abajo.
- *Sin banner de cookies* — la página no tiene aviso, o no se ha detectado ninguno.
- *Página no compatible* — pantallas internas del navegador (`chrome://` , la Chrome Web Store, páginas de error), donde ninguna extensión puede entrar.
- **Gestor de consentimiento.** El CMP identificado (Cookiebot, OneTrust, Didomi...), o *Ninguno* si el banner es casero. «Ninguno» no significa que no haya banner: significa que no es uno de los productos conocidos.
- **Consent Mode.** Si la web usa Google Consent Mode y de dónde se ha leído el dato:
- `gtag` — del estado interno de la etiqueta de Google. Es la fuente más fiable.

- `dataLayer` — de las llamadas `consent default` / `consent update` de la página.
- `red (gcs)` — deducido del parámetro `gcs` de las peticiones que la web ha enviado a Google. Es la prueba de lo que *realmente* ha viajado.
- **El desglose por categoría.** `ad_storage`, `analytics_storage`, `ad_user_data`, `ad_personalization` ... en verde si están concedidos y en rojo si están denegados. Esta es la parte útil para auditar: si has rechazado y aquí sigue apareciendo `granted`, ese sitio no está respetando tu decisión.

Los dos botones

- **Rechazar ahora.** Recarga la página para que la extensión lo intente otra vez. Es lo que hay que pulsar cuando un banner aparece tarde y se ha escapado al primer intento.
- **Permitir en este sitio.** Mete el dominio en la **lista blanca**: la extensión deja de tocar los avisos ahí y podrás contestarlos tú. Úsalo cuando rechazar rompa algo, o cuando quieras aceptar en una web concreta. El botón cambia a «**Sitio permitido (quitar)**» para deshacerlo. La lista blanca cubre también los subdominios: permitir `ejemplo.com` permite `tienda.ejemplo.com`.

Con el sitio permitido, el aviso se queda tal cual y esperando tu respuesta:



Los muros «consentir o pagar»

Cuando la extensión reconoce uno, la ventana emergente añade un recuadro de aviso con un interruptor «**Rechazar aquí de todas formas**». Está apagado a propósito: si lo enciendes, en ese sitio se pulsará el rechazo, con lo que eso implica (pasarela de pago o contenido bloqueado). Es una decisión por sitio y se puede volver a apagar.

El contador

Abajo, el total de banners rechazados desde que se instaló. Es informativo y solo cuenta los rechazos hechos en la pestaña principal, no en los marcos internos.

6. Preguntas frecuentes y problemas

En esta web no ha rechazado nada. ¿Está rota? No necesariamente. Cada banner es distinto y los caseros no siguen ningún estándar. Mira la ventana emergente: si dice *Banner detectado (sin rechazar)*, ha visto el gestor pero no ha encontrado el botón; hay que añadir su selector (ver la ficha técnica). Si dice *Muro consentir-o-pagar*, se ha parado a propósito.

¿Puede aceptar cookies por error? No. Solo busca botones de rechazo y hay una prueba automática que lo verifica poniéndole un botón «Aceptar todo» delante. Nunca lo pulsa.

He rechazado pero el desglose sigue diciendo `granted`. Eso es exactamente lo que la extensión sirve para descubrir. Dos causas posibles: el sitio no está aplicando tu rechazo (un problema suyo, y probablemente un incumplimiento), o la lectura viene de una llamada anterior al rechazo. Recarga con **Rechazar ahora** y vuelve a mirar: si tras la recarga sigue en `granted`, el problema es del sitio.

El aviso reaparece cada vez que entro. Pasa cuando la extensión ha tenido que ocultar el cartel sin poder pulsar el botón: al no registrarse ninguna decisión, la web vuelve a preguntar. También pasa si borras las cookies del navegador al cerrarlo.

Una web se ve rara o algo ha dejado de funcionar. Pon el sitio en la lista blanca con **Permitir en este sitio** y recarga. Si con eso se arregla, era la extensión; avísanos indicando el dominio.

¿Consume recursos? Muy poco. Durante los primeros catorce segundos de cada página revisa el DOM periódicamente en busca del banner y luego se detiene sola. En páginas con muchísimos elementos esa fase puede notarse ligeramente.

¿Funciona en ventana de incógnito? Solo si se lo permites: en `chrome://extensions`, Detalles → «Permitir en incógnito».

¿Y en Firefox? No. Necesita `world: "MAIN"` tal y como lo implementa Chromium.

¿Recopila datos míos? No. No hace ninguna petición de red. Todo (interruptor, lista blanca y contador) se guarda en tu navegador.

7. Seguridad: qué se ha revisado y qué se ha corregido

Se han leído íntegros el manifiesto, el service worker, los dos content scripts y la ventana emergente, y se ha probado la extensión cargada en un Chromium real con una batería de **30 comprobaciones automáticas**, todas en verde. Las correcciones aplicadas en esta revisión (versión **1.2.0** → **1.2.1**):

Corregido

1. Pulsaba botones del sitio que no tenían nada que ver con las cookies. `content/agent.js`. La lista `REJECT_SELECTORS` mezclaba selectores inequívocos de cada CMP con otros genéricos — `#denegar`, `[data-a="reject"]`, `button[mode="secondary"]`, `.qc-cmp2-summary-buttons button:first-child` — que se pulsaban **sin comprobar que estuvieran dentro de un aviso de cookies**. En la prueba, la versión anterior pulsó dos veces un botón llamado «Denegar acceso al panel» que era del sitio. En una tienda, un `button[mode="secondary"]` puede ser «Cancelar pedido». Los genéricos se han separado en `REJECT_SELECTORS_CTX` y ahora exigen que el botón cuelgue de un contenedor de cookies. *Es el fallo más grave de los encontrados.*

2. No rechazaba los banners en castellano bien escrito. `content/agent.js`. La lista de frases (`REJECT_TEXTS`) está sin tildes, pero `norm()` solo pasaba el texto a minúsculas: «**Sólo las necesarias**» no casaba con «**solo las necesarias**», ni «Únicamente necesarias» con «unicamente necesarias». Media lista estaba muerta para las webs españolas. `norm()` ahora quita los diacríticos (NFD) antes de comparar. En la prueba, la versión anterior dejaba el banner en pantalla y la corregida lo rechaza.

3. Una web podía mentir en la ventana emergente. `content/agent.js`. `detect.js` corre en el mundo MAIN, que es el contexto de la propia página, así que cualquier web puede publicar mensajes con la misma marca. El receptor los aceptaba tal cual y volcaba su contenido en el panel: un sitio podía escribir el texto que quisiera en «Gestor de consentimiento» o inventarse un estado de Consent Mode. Ahora se valida todo contra listas cerradas: el CMP solo puede ser uno de los once conocidos, las claves de Consent Mode solo las siete oficiales y los valores solo `granted` o `denied`. Comprobado con un intento real de inyección en la prueba automática. *Nota: no había riesgo de XSS —la ventana emergente ya usaba `textContent`, nunca `innerHTML`, con los datos—; el riesgo era de engaño al usuario.*

4. Podía romper los diálogos propios del sitio. `content/agent.js`. Tras pulsar, `unlockBody()` borraba **todos** los `.modal-backdrop` de la página y quitaba clases como `modal-open` del `<body>`. Combinado con el fallo 1, un clic equivocado podía dejar inutilizable el modal legítimo de una web. Ahora esa limpieza agresiva solo se hace cuando el botón pulsado estaba de verdad en un aviso de cookies; las superposiciones con nombre propio de CMP se siguen quitando siempre.

5. Fallo de JavaScript al arrancar la página. `content/agent.js`. Los scripts corren en `document_start`, cuando `document.body` todavía puede no existir, y había un `document.body.getAttribute('class')` sin protección: lanzaba una excepción que abortaba el primer intento de rechazo entero. Protegido, y toda la búsqueda de botones envuelta en `try/catch` por elemento, de modo que un elemento raro ya no anula el resto de la lista.

6. Lectura incorrecta del parámetro `gcs`. `content/detect.js`. La expresión regular era `/gcs=(G1[01g d]{2,3})/i`, con una `g` y un **espacio** sueltos dentro del grupo, y no contemplaba `-` ni `v`, que es como Google marca «sin definir». Resultado: con `gcs=G1--` no leía nada. Corregida, y ahora un carácter que no sea `0` ni `1` se deja **sin informar** en vez de darlo por denegado —antes se podía mostrar «denied» para algo que en realidad era «sin definir»—.

7. Estados inválidos de Consent Mode se mostraban tal cual. `content/detect.js`. `normVal()` devolvía sin filtrar cualquier valor raro que hubiera en el `dataLayer`. Ahora solo se admiten `granted` y `denied`, y las claves se contrastan con las siete oficiales.

8. El aviso de «consentir o pagar» se borraba solo. `content/agent.js`. El informe periódico (cada 700 ms) sobrescribía el estado `paywall` con `none` o `detected`, así que el panel acababa diciendo «Sin banner de cookies» en una web donde sí se había parado ante un muro de pago. Ahora `rejected` y `paywall` tienen prioridad y no se pisan.

9. Contador de rechazos con pérdida de cuenta. `background/service_worker.js`. El incremento era leer-modificar-escribir sin serializar: con varias pestañas rechazando a la vez se perdían incrementos. Ahora las escrituras se encadenan.

10. Comprobación del remitente en los mensajes internos. `background/service_worker.js` y `content/agent.js`. Se ha añadido `sender.id !== chrome.runtime.id` en los receptores de `chrome.runtime.onMessage`. Es defensa en profundidad: la extensión no declara `externally_connectable`, así que hoy no puede recibir mensajes de fuera, pero el guardián evita que un cambio futuro lo abra.

11. `host_permissions` redundante y dependencia innecesaria de `tab.url`. `manifest.json` y `popup/popup.js`. Se ha quitado la declaración `"host_permissions": ["<all_urls>"]` y la ventana emergente ha dejado de depender de `tab.url`: el dominio se lo pide ahora al propio content script. Ver la nota honesta más abajo.

12. Ortografía de los textos que lee el usuario. `popup/popup.html`, `popup/popup.js`, `manifest.json` y `README.md`: «activaría», «así», «automáticamente», «aquí», «Sí», «categoría», «Página». Las frases de `REJECT_TEXTS` de `agent.js` **se han dejado a propósito sin tildes**, porque son patrones de comparación, no texto para leer; ponerles tildes las rompería. Queda avisado en el propio código y en el README para que nadie las «corrija» por error.

Revisado y correcto de origen

- **Sin ejecución de código remoto.** No hay `eval`, ni `new Function`, ni scripts externos. Todo el código viaja en la extensión.
- **Sin XSS.** El único `innerHTML` es `box.innerHTML = ''` (vaciar un contenedor). Todo el contenido se inserta con `createElement` y `textContent`.
- **Sin peticiones de red.** Ni una. No hay servidor, ni analítica, ni telemetría.
- **Sin credenciales ni claves** escritas en el código.
- **Sin `var_dump` ni volcados** de depuración vivos.
- **Los mensajes entre marcos** ya comprobaban `e.source !== window`, de modo que un `iframe` de terceros no puede inyectar estado en la página que lo contiene.
- **La lista blanca** compara bien los subdominios (`host === w || host.endsWith('.') + w`), sin el fallo clásico de que `malaejemplo.com` cuele como `ejemplo.com`.
- **Las llamadas a la API del navegador** que pueden fallar porque la pestaña se cerró ya estaban envueltas (`safeAction`).

Pendiente, y por qué no se ha tocado

Una web siempre podrá falsear su propio informe. Es una limitación de diseño que **no tiene arreglo** en Manifest V3, y conviene entenderla. Para leer `window.dataLayer` y los objetos del CMP hay que ejecutar código en el «mundo MAIN», que es el mismo contexto de JavaScript de la página. No existe ningún canal privado entre ese mundo y el aislado: cualquier mensaje que uno mande, la web puede leerlo e imitarlo. La

corrección 3 acota el daño —una web ya solo puede mentir con valores de una lista cerrada, no escribir texto propio en el panel— pero un sitio determinado a engañarte podría declararse «Cookiebot» con todo «denied» sin serlo. **Para auditar en serio, contrasta siempre con la fila red (gcs)**, que se deduce de las peticiones realmente enviadas a Google y es mucho más difícil de falsificar.

El acceso a todos los sitios no se puede reducir. Se quitó `host_permissions: ["<all_urls>"]` del manifiesto, pero hay que decirlo con claridad: **el acceso efectivo no ha bajado**. Se comprobó en la prueba y en Manifest V3 los `matches` de `content_scripts` conceden por sí solos ese mismo acceso de host; tras el cambio, el service worker seguía pudiendo hacer una petición a otro dominio. Lo que se ha eliminado es una **declaración redundante**, no un privilegio. Una extensión cuyo trabajo es rechazar avisos de cookies en cualquier web necesita `<all_urls>` por definición, y el aviso que Chrome enseña al instalarla seguirá siendo el mismo.

La cobertura de banners es la que es. No hay forma de cubrir todos los avisos caseros del mundo. Los CMP grandes están soportados; el resto depende de la heurística por texto. Añadir un banner nuevo es añadir su selector (ver la ficha técnica).

El repaso periódico cuesta algo de rendimiento. Durante 14 segundos, cada 700 ms, se recorre el DOM buscando el banner y se evalúa el detector de muros de pago. En páginas muy grandes puede notarse. Reducirlo cambiaría el comportamiento —dejaría de cazar los banners que aparecen tarde—, así que se ha dejado como estaba.

8. Ficha técnica

Ficheros

Fichero	Papel
<code>manifest.json</code>	Manifest V3. Versión 1.2.1, mínimo Chrome 111.
<code>background/service_worker.js</code>	Insignia ✓ por pestaña y contador global.
<code>content/detect.js</code>	Content script en mundo MAIN . Detecta el CMP y el Consent Mode; rechaza por API.
<code>content/agent.js</code>	Content script aislado . Configuración, rechazo por DOM, validación y estado.
<code>popup/popup.html</code> · <code>popup.js</code> · <code>popup.css</code>	La ventana emergente.
<code>icons/</code>	Iconos de 16, 48 y 128 px.
<code>README.md</code>	Notas para quien mantenga el código.

Permisos declarados

`storage`, `activeTab`. **No** hay `host_permissions`. Los content scripts se declaran con `matches: ["<all_urls>"]`, `run_at: "document_start"` y `all_frames: true` (los banners a menudo viven en un `iframe`).

Cómo encaja todo

`detect.js` corre en el contexto de la página, único sitio desde el que se ven `window.dataLayer`, `window.google_tag_data.ics` y los objetos del CMP. Publica su informe con `window.postMessage`, marcado con la clave `__ecomnc`. `agent.js`, en el mundo aislado, es el que tiene acceso a `chrome.storage`: lee la configuración, decide si actuar, pulsa el botón de rechazo en el DOM, ordena a `detect.js` el rechazo por API, **valida** lo que le llega y responde al popup cuando este pide `getStatus`. `service_worker.js` solo pinta la insignia y lleva la cuenta.

Ambos scripts reintentan cada 700 ms hasta 21 veces (unos 14 segundos), porque muchos CMP cargan de forma asíncrona. `agent.js` para en cuanto consigue pulsar.

Configuración guardada (`chrome.storage.local`)

Clave	Valor	Qué es
<code>enabled</code>	<code>true</code> / <code>false</code>	Interruptor general.
<code>whitelist</code>	array de dominios	Sitios donde no actúa. Cubre subdominios.
<code>payMode</code>	<code>'skip'</code> / <code>'reject'</code>	Qué hacer en los muros «consentir o pagar». <code>skip</code> por defecto.
<code>totalRejected</code>	número	Contador acumulado.

Rechazo por API, por CMP

CMP	Llamada
Cookiebot	<code>Cookiebot.decline()</code>
OneTrust	<code>OneTrust.RejectAll()</code>
Didomi	<code>Didomi.setUserDisagreeToAll()</code>
Usercentrics	<code>UC_UI.denyAllConsents()</code> / <code>rejectAllConsents()</code>
Osano	<code>Osano.cm.denyAll()</code>
Complianz	<code>cmplz_deny_all()</code>
CookiesPlus	<code>cookieGdpr.rejectAllCookies()</code>

Añadir un banner que no se rechaza

En `content/agent.js`:

- Si el botón tiene un selector **inequívoco y exclusivo** de ese CMP → a `REJECT_SELECTORS`.
- Si el selector es genérico y podría existir en otros botones de la web → a `REJECT_SELECTORS_CTX`, donde se exige contexto de cookies antes de pulsar. **Ante la duda, siempre a esta segunda lista.**
- Si lo que identifica al botón es su texto → a `REJECT_TEXTS`, **sin tildes y en minúsculas** (`norm()` normaliza el texto de la página antes de comparar).
- Si el cartel no se cierra solo tras pulsar → su contenedor a `BANNER_CONTAINERS`.

Sin registros

La extensión no escribe ficheros de log: no puede, no tiene acceso al disco. Para depurar, la consola de la página (F12) para los content scripts, y en `chrome://extensions` el enlace «service worker» para el de fondo.

Prueba automática

El trabajo se verificó con un guion de Playwright que carga la extensión en un Chromium real contra una página con aviso de cookies, Google Consent Mode simulado y dos botones trampa del sitio. Comprueba 30 puntos: que rechaza, que no acepta, que no toca lo que no debe, el estado y el Consent Mode que llegan al panel, la insignia, el contador, los permisos, el intento de falseo, la lista blanca, el interruptor general y que no hay errores de JavaScript. Ejecutado contra la versión 1.2.0 original, falla en el punto 2 (no rechazaba y pulsaba el botón equivocado); contra la 1.2.1, pasa entero.